

两条链表操作举例

归并有序线性表算法

【例2】已知线性表LA和LB按值非递减排列，现要将LA和LB归并成一个新的线性表LC，且LC中元素仍按值非递减排列。
【粗略的算法】逐个插入到LC中。



已知线性表LA和LB按值非递减排列，现要将LA和LB归并成一个新的线性表LC，且LC中元素仍按值非递减排列。
【粗略的算法】逐个插入到LC中。

如：LA = (7, 15, 29, 38), LB = (3, 7, 46, 57)
LC = (3, 7, 7, 15, 29, 38,)

Diagram illustrating the merging process:

- LA = (7, 15, 29, 38)
- LB = (3, 7, 46, 57)
- LC = (3, 7, 7, 15, 29, 38,)

Arrows indicate the insertion of elements from LA and LB into LC. The element 46 from LB is circled, and an arrow points to the next empty space in LC.

归并有序线性表算法

【算法思路】

`InitList()`

1. [初始化] 构造空线性表LC，设置3个辅助指针i、j和k，分别指向LA、LB的第一个元素以及LC的最后一个元素；

`ListLength()`

2. [两两比较] 循环，直到其中一个线性表的元素全部处理完毕：

① 读取LA中的第i个元素和LB中的第j个元素 `GetElem()`

② 若前者较小，则将前者插入到LC的表尾，i和k相应后移一个位置；
否则将后者插入到LC的表尾，j和k相应后移一个位置；

3. [扫尾]

`ListInsert()`

① 若LA中还有剩余元素，则将LA的剩余元素全部插入到LC的表尾

② 若LB中还有剩余元素，则将LB的剩余元素全部插入到LC的表尾

↘ 归并两个有序单链表

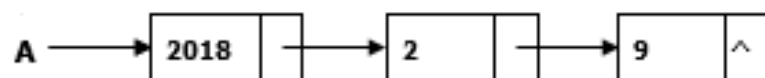
```
struct node * MergeList_L(struct node *La, struct node * Lb) {  
    struct node * pa = La, * pb = Lb; //不带头结点  
    struct node * Lc = NULL;  
    if (!pa && !pb) return Lc;  
    else if (!pa) { Lc=pb; return Lc; }  
    else if (!pb) { Lc=pa; return Lc; }  
    else if (pa->date <= pb->date)  
        { Lc=pc=pa; pa=pa->next; }  
    else { Lc=pc=pb; pb=pb->next; }  
}
```

↘ 归并两个有序单链表

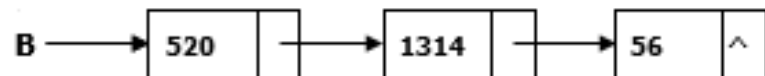
```
while ( pa && pb ) {    // 归并
    if ( pa->date <= pb->date )
        { pc->next = pa; pc = pa; pa = pa->next; }
    else { pc->next = pb; pc = pb; pb = pb->next; }
}
-----
pc->next = pa ? pa : pb;    // 插入剩余段
return Lc;
} // MergeList_L
-----
```

例2：两条单链表的组合

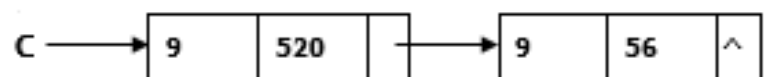
链表 A 存储了一批非负整数，如下所示 (^ 表示链表尾)：



链表 B 也存储了一批非负整数，如下所示 (^ 表示链表尾)：



编写一个名为 `coprime` 的函数，形参 A, B 为上述链表，函数返回一个新链表 C，C 中包含了所有满足下述条件的组合：(a) 从 A, B 中各取一个数，(b) 两个数的最大公约数为 1 (即互质)。上例中，执行 `C=coprime(A, B)` 后结果如下 (结点顺序随意)：



要求：(1) 给出两种链表结点 `struct L1` 和 `struct L2` 的定义

(2) 编写 `coprime` 函数，实现题目要求的功能，允许编写其它函数供 `coprime` 函数调用

```
struct L1 { //L1 链表结点定义
    int d;
    struct L1 *next;
};
struct L2 { //L2 链表结点定义
    int d1, d2;
    struct L2 *next;
};
int gcd(int a, int b) { //最大公约数
    int t;
    while (a != 0) {
        t = a;
        a = b % a;
        b = t;
    }
    return t;
}
```




```
struct L2* coprime(struct L1* A, struct L1* B) {  
  
    struct L2* C = NULL, *temp1, *temp; //C 初始化为 NULL  
    struct L1* p = A, *q;  
    while (p != NULL) { //外重循环（必须有 p=p->next）  
        q = B; //q 指向链表头  
        while (q != NULL) { //内重循环（必须有 q=q->next）  
            if (gcd(p->d, q->d) == 1) {  
                temp = (struct L2*)malloc(sizeof(struct L2));  
  
                temp->d1 = p->d;  
                temp->d2 = q->d;  
                temp->next = NULL;  
                if (C == NULL) //插入结点  
                    C = temp;  
                else  
                    temp1->next = temp;  
                temp1 = temp;  
            }  
            q = q->next;  
        }  
        p = p->next;  
    }  
  
    return C;  
}
```